

Student

We covered ranking functions last time. What is the next family of window functions, and what new problems do they solve?

ChatGPT • AI Assistant

CONCEPT CARD

This topic covers two related but distinct capabilities, both built on the exact same **OVER (PARTITION BY ... ORDER BY ...)** foundation from SQL 1.13:

1. **LAG and LEAD** - analytic functions that let a row 'reach' into a neighboring row's value. LAG reaches backward (a previous row), LEAD reaches forward (a following row). These answer questions like 'what was yesterday's value' or 'how many days until the next event.'
2. **The window frame clause** - controls exactly which set of rows a function like SUM or AVG operates on relative to the current row.

This is what powers running totals, cumulative sums, and moving averages - and it is the most powerful, most misunderstood part of window functions.

🗣️ ANALOGY

LAG and LEAD are like tapping the person standing directly behind or in front of you in a line and asking 'what number did you get?' The frame clause is different - it is like being handed a sliding window you can look through that shows 'everyone from the start of the line up to me' (running total) or 'just the 7 people nearest to me' (moving average), and the window slides forward as you move down the line.

🌐 REAL WORLD APPLICATION

LAG/LEAD power period-over-period comparisons in every revenue dashboard - day-over-day, week-over-week, month-over-month growth. Running totals and moving averages power cumulative revenue charts, smoothed trend lines, and threshold-crossing detection (e.g., 'at which order did this customer become a \$10,000 lifetime customer').

Nearly every analytics dashboard you have ever looked at uses one of these patterns under the hood.

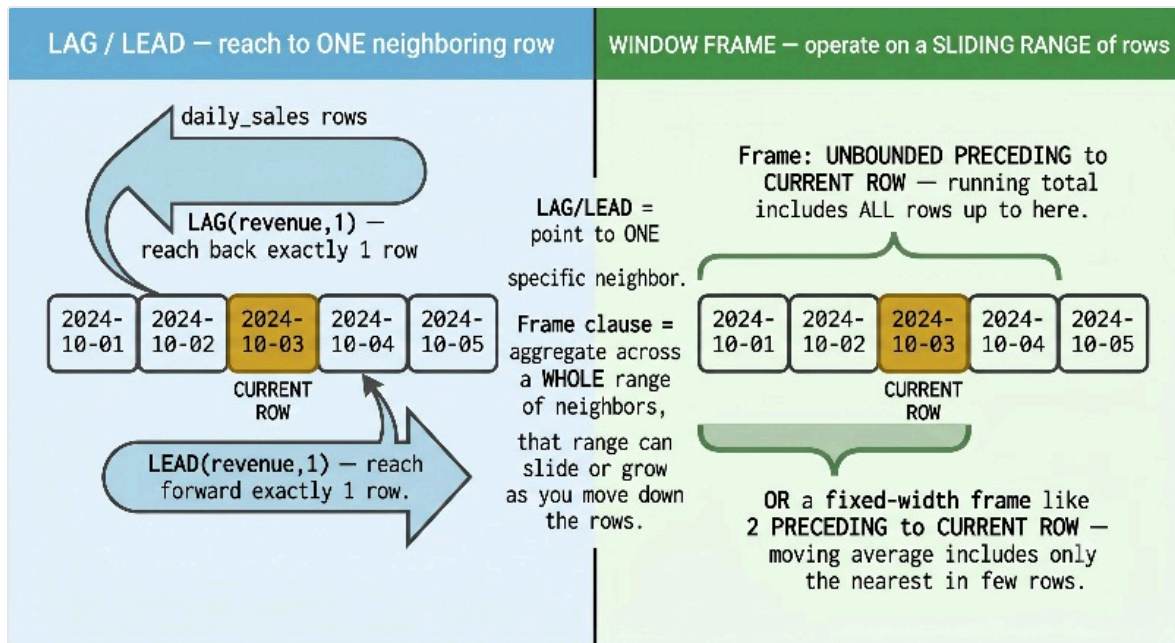
What schema are we using?

ChatGPT • AI Assistant

CONCEPT CARD

A daily sales summary table for the core examples, plus the familiar orders table for the practice problems:

- `daily_sales(sale_date, revenue)` - one row per calendar day, total revenue that day
- `orders(order_id, customer_id, order_date, total_amount, status)` - used for the next-order-gap example and both practice problems



LAG and LEAD vs Window Frame — Two Different Mental Models

LAG — REACHING BACK TO A PREVIOUS ROW

ChatGPT • AI Assistant

A horizontal sequence of 4 date-labeled boxes representing daily_sales rows in chronological order: Jan 1 (revenue=12000), Jan 2 (revenue=12500), Jan 3 (revenue=11800), Jan 4 (revenue=13200). Below the Jan 2 box, a backward-pointing arrow labeled 'LAG(revenue,1,0)' points to the Jan 1 box, with the value '12000' copied down into a new row labeled 'previous_day_revenue' beneath Jan 2. Similarly for Jan 3 (arrow to Jan 2, copying 12500) and Jan 4 (arrow to Jan 3, copying 11800). The Jan 1 box itself has NO backward arrow (there is nothing before it) — instead a dashed box appears to its left labeled 'NO PRIOR ROW EXISTS' with a dotted arrow showing the DEFAULT value '0' being inserted into Jan 1's previous_day_revenue field instead of NULL. A warning annotation beneath Jan 1's row reads: 'Default value (0) fills the boundary gap for the FIRST row — but watch out: if this 0 then becomes a denominator in a percentage formula, the math breaks down.' Clean horizontal sequence, arrows clearly showing the 1-row backward reach, boundary condition visually distinguished with dashed styling.

LAG — Reaching Back with Offset and Default, Including the Boundary Row

Student

Let's start with LAG. What exactly does it do and what are those extra arguments - offset and default?

ChatGPT • AI Assistant

CONCEPT CARD

`LAG(column, offset, default)` returns the value of `column` from a row **before** the current row, within the same partition, based on the ORDER BY.

- **column** - which value you want to pull from the earlier row.
- **offset** - how many rows back to look. Defaults to **1** (the immediately preceding row) if omitted.
- **default** - the value to return when there is no row that far back (e.g., the very first row in a partition has no 'previous' row). Defaults to **NULL** if omitted.

sql — Code Example

```
LAG(column_name, offset, default_value) OVER (  
    PARTITION BY ...  
    ORDER BY ...  
)
```

EXAMPLE CARD

Worked example: put yesterday's revenue next to today's, then compute day-over-day change and growth percentage - all in one query.

sql — Code Example

```
SELECT sale_date,
       revenue,
       LAG(revenue, 1, 0) OVER (ORDER BY sale_date) AS
previous_day_revenue,
       revenue - LAG(revenue, 1, 0) OVER (ORDER BY sale_date)
       AS day_over_day_change,
       ROUND(
         (revenue - LAG(revenue, 1, 0) OVER (ORDER BY sale_date))
         / LAG(revenue, 1, 0) OVER (ORDER BY sale_date) * 100,
         2
       ) AS growth_pct
FROM   daily_sales
ORDER BY sale_date;
```

Sample output:

code – Code Example				
sale_date	revenue	previous_day_revenue	day_over_day_change	growth_pct
2024-01-01	12000	0	12000	
NULL ← division by 0				
2024-01-02	12500	12000	500	4.17
2024-01-03	11800	12500	-700	-5.60
2024-01-04	13200	11800	1400	11.86

Notice the very first row: `LAG(revenue, 1, 0)` returns **0** - the default we explicitly supplied - instead of NULL, because there is no row before 2024-01-01 in this dataset. The `growth_pct` column for that first row is still NULL because dividing by the default 0 produces a division-by-zero issue (handled by the database returning NULL or erroring, depending on platform) - choosing a default of 0 fixed the 'previous`day`revenue' column but did not magically solve the percentage-growth math for a day with no real prior value.

 WARNING

Choosing 0 as the default for LAG can silently corrupt percentage-growth calculations - a 0 in the denominator either errors or produces NULL/infinity depending on the database, which is often *correct* behavior (there genuinely is no

'growth percentage' from a non-existent baseline), but it is easy to misinterpret a resulting NULL as 'no growth' rather than 'no valid comparison exists.' For absolute differences (`day_over_day_change`), defaulting to 0 is usually safe and intuitive. For ratios and percentages, consider leaving the default as NULL (the function's own default) so that the absence of a true baseline is clearly visible rather than silently producing a misleading number.

sql – Code Example

```
-- SAFER for percentage calculations: let LAG default to NULL,  
-- and let the NULL propagate naturally through the arithmetic  
SELECT sale_date,  
       revenue,  
       LAG(revenue) OVER (ORDER BY sale_date) AS previous_day_revenue,  
       ROUND(  
           (revenue - LAG(revenue) OVER (ORDER BY sale_date))  
           / LAG(revenue) OVER (ORDER BY sale_date) * 100,  
           2  
       ) AS growth_pct  
FROM   daily_sales  
ORDER BY sale_date;  
-- The very first row's growth_pct is honestly NULL – 'not applicable'  
--  
-- rather than implying a 0 baseline that never actually existed.
```

CONCEPT CARD

Using a larger offset - comparing to 7 days ago instead of 1 day ago:

sql – Code Example

```
SELECT sale_date,  
       revenue,  
       LAG(revenue, 7) OVER (ORDER BY sale_date) AS  
revenue_7_days_ago,  
       revenue - LAG(revenue, 7) OVER (ORDER BY sale_date) AS  
week_over_week_change  
FROM   daily_sales  
ORDER BY sale_date;  
-- offset=7 reaches back exactly 7 rows, enabling week-over-week  
-- comparisons instead of day-over-day, using the SAME function,  
-- just a different offset argument.
```

INTERVIEW CARD

Frequency: Very High

Question: What do the three arguments of LAG() represent, and what happens at the boundary of a partition?

Expected Answer: LAG(column, offset, default) - column is the value to retrieve from an earlier row; offset is how many rows back to look, defaulting to 1 if omitted; default is the value returned when no row exists that far back, defaulting to NULL if omitted. At the start of a partition (or the whole result set if no PARTITION BY is used), there is no preceding row for the first few rows depending on the offset - LAG returns the specified default value (or NULL) for those boundary rows instead of erroring, which is essential to know because downstream arithmetic on a NULL will itself produce NULL, silently propagating through calculations like day-over-day change.

Follow-up: Why might supplying a default of 0 be appropriate for an absolute difference calculation but risky for a percentage growth calculation? (Answer: For an absolute difference like `revenue - LAG(revenue, 1, 0)`, a 0 baseline produces a sensible 'full revenue counted as the change' on day one.

For a percentage calculation like `(revenue - prev) / prev * 100`, a 0 baseline causes division by zero, which can return NULL, error, or infinity depending on the database - potentially masking the fact that no real comparison baseline exists, which is better represented by an honest NULL than by an artificial 0.)

🔗 PRACTICE QUIZ

1. What does LAG(revenue, 3) return for the 2nd row of a partition (assuming no default is specified)?
2. What is the default value of the offset argument if you write just LAG(revenue)?
3. Why might you choose NOT to supply a default value of 0 when using LAG to compute a percentage growth?
4. How would you modify a day-over-day LAG query to instead compare each row to the value 7 rows earlier?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. NULL - the 2nd row of a partition has only 1 row before it, but offset=3 asks for 3 rows back, which does not exist; with no default specified, LAG falls back to its own default of NULL.

2. 1 - LAG(revenue) with no second argument behaves identically to LAG(revenue, 1), reaching back exactly one row.
3. Because a 0 default becomes the denominator in a percentage formula, causing division-by-zero issues (NULL, error, or infinity depending on platform) that can obscure the fact that no genuine prior value exists to compare against - an honest NULL is usually clearer than a misleading default.
4. Change the offset argument from 1 (or omitted) to 7: `LAG(revenue, 7) OVER (ORDER BY sale_date)`.

🔍 CHECKPOINT QUESTION

Write a query using LAG that shows each order's *totalamount alongside the SAME customer's previous order's totalamount*, with a default of NULL (not 0), partitioned correctly so that one customer's orders never leak into being compared against a different customer's previous order.

LEAD — REACHING FORWARD TO A FOLLOWING ROW

ChatGPT • AI Assistant

A horizontal sequence of 4 order boxes for customer 101, arranged by order_date left to right: Jan 5 (order 5001), Feb 12 (order 5034), Apr 30 (order 5210), Sep 15 (order 5402). Below the Jan 5 box, a forward-pointing arrow labeled 'LEAD(order_date)' points to the Feb 12 box, with the date 'Feb 12' copied down into a new field labeled 'next_order_date' beneath Jan 5, alongside a computed '38 days' gap. Similarly for Feb 12 (arrow to Apr 30, gap=77 days) and Apr 30 (arrow to Sep 15, gap=138 days). The Sep 15 box (rightmost, highlighted orange) has NO forward arrow — instead a dashed box appears to its right labeled 'NO FOLLOWING ROW EXISTS YET' with the next_order_date field showing 'NULL' in orange text. An annotation beneath reads: 'This NULL is NOT a bug — it correctly signals this is the customer's MOST RECENT order. Filtering WHERE next_order_date IS NULL is a valid technique for isolating each customer's latest row.' Clean horizontal sequence mirroring the LAG figure's style but with forward-pointing arrows and the boundary condition at the END of the sequence instead of the beginning.

LEAD — Reaching Forward, and the Honest NULL at the Last Row

Student

LEAD must be the mirror image then - looking forward instead of back?

ChatGPT • AI Assistant

CONCEPT CARD

Exactly. `LEAD(column, offset, default)` has identical arguments and identical behavior to LAG, except it reaches **forward** to a following row instead of backward to

a preceding one.

sql – Code Example

LEAD(column_name, offset, default_value) OVER (
 PARTITION BY ...
 ORDER BY ...
)

EXAMPLE CARD

Worked example: compute the gap in days until each customer's NEXT order - a classic churn/retention metric.

sql – Code Example

SELECT order_id,
 customer_id,
 order_date,
 total_amount,
 LEAD(order_date) OVER (
 PARTITION BY customer_id
 ORDER BY order_date
) AS next_order_date,
 LEAD(order_date) OVER (
 PARTITION BY customer_id
 ORDER BY order_date
) - order_date AS days_until_next_order
FROM orders
WHERE status ≠ 'cancelled'
ORDER BY customer_id, order_date;

Sample output (customer 101 with 4 orders):

code – Code Example

order_id	customer_id	order_date	next_order_date	days_until_next_order
5001	101	2024-01-05	2024-02-12	38
5034	101	2024-02-12	2024-04-30	77
5210	101	2024-04-30	2024-09-15	138
5402	101	2024-09-15	NULL	NULL

← their MOST RECENT order – no next one yet

Notice the last row: **LEAD** correctly returns **NULL** for **next_order_date**, because there is no order after this one (yet) for this customer - this is the **most recent order** in their history, and NULL here is the entirely correct, honest signal of that fact rather than a data quality problem to fix.

CONCEPT CARD

A direct business use of this exact pattern - flagging customers at risk of churn:

sql – Code Example

```
WITH order_gaps AS (  
    SELECT order_id, customer_id, order_date,  
           LEAD(order_date) OVER (  
               PARTITION BY customer_id ORDER BY order_date  
           ) AS next_order_date  
    FROM   orders  
    WHERE  status ≠ 'cancelled'  
)  
SELECT customer_id,  
       order_date          AS last_known_order,  
       CURRENT_DATE - order_date AS days_since_this_order  
FROM   order_gaps  
WHERE  next_order_date IS NULL      -- this IS each customer's most  
      recent order  
      AND CURRENT_DATE - order_date > 90 -- and it has been over 90  
      days since  
ORDER BY days_since_this_order DESC;  
-- Returns: customers whose LAST order was over 90 days ago – likely  
churned
```

This combines a window function (**LEAD**) with a WHERE filter in the **outer** query - exactly the CTE pattern from SQL 1.13, since **next_order_date** cannot be filtered directly in the same query level where it is computed.

INTERVIEW CARD

Frequency: Very High

Question: What does it mean when LEAD() returns NULL for a row, and is that always a data quality issue?

Expected Answer: LEAD() returns NULL (or the specified default) when there is no row following the current row within its partition according to the ORDER BY - most commonly because the current row is the LAST row in that partition. This is not a data quality issue at all in most cases; it is the correct, expected signal that 'there is

no next event yet' - for example, a customer's most recent order has no 'next order date' simply because it has not happened yet.

Treating this NULL as meaningful (e.g., 'this row represents the most recent order in this customer's history') is itself a common and valid analytical pattern, often used precisely to identify the latest record per group.

Follow-up: How would you use LEAD's NULL result to identify only the single most recent order for every customer, without using *ROWNUMBER* or *RANK* at all?

(Answer: Filter *WHERE nextorder_date IS NULL* in the outer query - since LEAD only returns NULL for the last row in each customer's partition by definition, this directly isolates exactly one row per customer: their most recent order.)

🔗 PRACTICE QUIZ

1. What is the key structural difference between LAG and LEAD?
2. In the order-gap example, what does a NULL in *nextorderdate* tell you about that specific row?
3. Could you use LEAD's NULL behavior to find each customer's most recent order, as an alternative to *ROW_NUMBER*?
4. Why must the churn-detection query filter on *nextorderdate IS NULL* in an outer query rather than directly alongside the LEAD computation?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. LAG reaches backward to a preceding row in the partition; LEAD reaches forward to a following row. Both share the identical (column, offset, default) argument structure and *OVER()* syntax.
2. It tells you this row is the LAST row in its partition by the given *ORDER BY* - there is no subsequent order for that customer in the dataset, meaning this is their most recent order to date.
3. Yes - *WHERE next_order_date IS NULL* after computing *LEAD(orderdate) PARTITION BY customerid ORDER BY order_date* directly isolates each customer's most recent order, since LEAD only produces NULL for the last row in each partition.
4. Because LEAD is a window function computed at the *SELECT* step of SQL's logical execution order, which runs after *WHERE* - exactly the same restriction covered for

ranking functions in SQL 1.13. The fix is identical: compute it in a CTE, filter on the result in the outer query.

❓ CHECKPOINT QUESTION

Write a query that, for every order, shows the number of days SINCE the previous order from the same customer (using LAG) AND the number of days UNTIL the next order from the same customer (using LEAD), in the same SELECT statement. Then explain what both columns would show for a customer's very first order and very last order respectively.

THE WINDOW FRAME CLAUSE — RUNNING TOTALS WITH ROWS BETWEEN

ChatGPT • AI Assistant

A four-row vertical sequence diagram showing daily_sales rows Jan 1 through Jan 4, each with a bracket showing exactly which rows are included in that row's frame. Row Jan 1: a bracket spans ONLY the Jan 1 box itself, labeled 'Frame: [Jan 1] — SUM = 12000.' Row Jan 2: a bracket spans Jan 1 AND Jan 2, labeled 'Frame: [Jan 1, Jan 2] — SUM = 24500.' Row Jan 3: a bracket spans Jan 1 through Jan 3, labeled 'Frame: [Jan 1..Jan 3] — SUM = 36300.' Row Jan 4: a bracket spans Jan 1 through Jan 4 (the full set so far), labeled 'Frame: [Jan 1..Jan 4] — SUM = 49500.' Each successive bracket is drawn slightly longer than the one above it, visually emphasizing the GROWING nature of the frame as you move down the rows. A side annotation reads: 'UNBOUNDED PRECEDING = frame always starts at the very first row of the partition. CURRENT ROW = frame always ends at wherever we currently are. The frame never shrinks — it only grows by one row at a time.' Clean vertical layout with progressively longer brackets, consistent left-alignment showing the fixed starting point (UNBOUNDED PRECEDING) and the moving ending point (CURRENT ROW).

Running Total — The Growing Frame, Row by Row

Student

Now the part you said was the most powerful and most misunderstood - the frame clause. What is actually happening when I add ORDER BY inside OVER() for something like SUM?

ChatGPT • AI Assistant

CONCEPT CARD

This is the critical insight: **the moment you add ORDER BY inside OVER(), an aggregate function like SUM or AVG no longer operates on the WHOLE partition - it operates on a FRAME, a specific subset of rows relative to the current row, that changes as you move from row to row.**

By default, when ORDER BY is present and no explicit frame is written, the frame is:

code – Code Example

RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

This default frame means: 'every row from the very start of the partition, up to and including the current row.' This is **exactly** what produces a running total.

sql – Code Example

```
SELECT sale_date,
       revenue,
       SUM(revenue) OVER (ORDER BY sale_date) AS running_total
FROM   daily_sales
ORDER BY sale_date;
-- Equivalent, written explicitly:
SELECT sale_date,
       revenue,
       SUM(revenue) OVER (
           ORDER BY sale_date
           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_total
FROM   daily_sales
ORDER BY sale_date;
```

EXAMPLE CARD

Tracing a running total by hand, row by row:

code – Code Example

sale_date	revenue	running_total	
2024-01-01	12000	12000	← frame = [Jan 1] only
→ SUM = 12000			
2024-01-02	12500	24500	← frame = [Jan 1, Jan 2]
→ SUM = 12000+12500			
2024-01-03	11800	36300	← frame = [Jan 1..Jan 3]
→ SUM = 24500+11800			
2024-01-04	13200	49500	← frame = [Jan 1..Jan 4]
→ SUM = 36300+13200			

At every row, the frame **grows by exactly one row** - it always starts at the very first row of the partition (**UNBOUNDED PRECEDING**) and always ends at the current row (**CURRENT ROW**). This growing-frame behavior is precisely what a running/cumulative total means.

CONCEPT CARD

Combining a running total with PARTITION BY - resetting the cumulative sum per customer:

sql – Code Example

```
SELECT order_id,
       customer_id,
       order_date,
       total_amount,
       SUM(total_amount) OVER (
         PARTITION BY customer_id
         ORDER BY order_date
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_customer_spend
FROM   orders
WHERE  status ≠ 'cancelled'
ORDER BY customer_id, order_date;
```

Output (two customers shown):

code – Code Example

order_id	customer_id	order_date	total_amount	running_customer_spend
5001	101	2024-01-05	850.00	850.00
5034	101	2024-02-12	1200.00	2050.00
5210	101	2024-04-30	920.00	2970.00
6001	102	2024-01-10	3000.00	3000.00 ←
RESETS for customer 102				
6045	102	2024-03-22	1500.00	4500.00

Notice the running total correctly **resets to start fresh** the moment the partition changes from customer 101 to customer 102 - PARTITION BY ensures the frame's **UNBOUNDED PRECEDING** boundary never crosses into a different customer's rows.

⚠ WARNING

A frequent mistake: forgetting that without an explicit ORDER BY inside OVER(), an aggregate function defaults to operating on the ENTIRE partition for every row - not a running total.

sql – Code Example

```
-- WITHOUT ORDER BY: this is NOT a running total – it's the SAME grand
total
-- repeated on every single row, because the default frame with no
ORDER BY
-- is the WHOLE partition, not a growing slice of it.
SELECT sale_date, revenue,
       SUM(revenue) OVER () AS total_all_time  -- same number on
every row!
FROM   daily_sales;

-- WITH ORDER BY: NOW it becomes a running total, growing row by row
SELECT sale_date, revenue,
       SUM(revenue) OVER (ORDER BY sale_date) AS running_total
FROM   daily_sales;
```

The presence or absence of ORDER BY inside OVER() is the single switch that turns a window aggregate from 'grand total repeated everywhere' into 'running total that grows as you move down the rows.'

INTERVIEW CARD

Frequency: Very High

Question: What is the default window frame when ORDER BY is present inside OVER(), and why does this default produce a running total?

Expected Answer: The default frame when ORDER BY is specified is RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW - meaning the frame includes every row from the start of the partition up through the current row. Because this frame grows by including one additional row each time you move to the next row in the ORDER BY sequence, an aggregate function like SUM computed over this frame naturally accumulates - producing a running total.

Without ORDER BY, there is no defined sequence of 'rows up to here,' so the default frame becomes the entire partition for every single row, producing the same grand total repeated on every row instead of an accumulating value.

Follow-up: What happens to the running total's reset point if you add PARTITION BY customer_id to this query? (Answer: The UNBOUNDED PRECEDING boundary of the frame becomes scoped to each partition individually - the running total restarts from zero at the first row of every new customer's group, never accumulating across customer boundaries, exactly like how PARTITION BY resets ranking functions per group.)

PRACTICE QUIZ

1. What is the default frame when ORDER BY is present but no explicit frame is written?
2. What does SUM(revenue) OVER () with no ORDER BY and no PARTITION BY return on every row?
3. Why does adding ORDER BY alone (without any explicit ROWS BETWEEN clause) turn SUM into a running total?
4. If you PARTITION BY customer_id in a running total query, where does the running total reset?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 **ANSWER KEY**

1. RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW.
2. The grand total of the ENTIRE result set (or entire partition, if PARTITION BY is also present) - the same single number repeated identically on every output row, because with no ORDER BY there is no 'rows up to here' concept, so the default frame is the whole set.
3. Because the presence of ORDER BY activates the default frame RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW, which grows by one row at a time as the ORDER BY sequence advances - an accumulating frame fed into SUM is exactly the definition of a running total.
4. At the first row of every new value of customer_id - PARTITION BY scopes the UNBOUNDED PRECEDING boundary to within each partition, so the accumulation never crosses from one customer's rows into another's.

🔗 **CHECKPOINT QUESTION**

Write a query that computes the running total of order count (not revenue) per customer, ordered by order_date, using ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW explicitly written out. Then explain in one sentence what the running total represents for the very last row of each customer's partition.

MOVING WINDOWS, AND ROWS VS RANGE WITH DUPLICATE VALUES

A two-panel comparison diagram showing the identical 4-row dataset (order 1 on Jan 1, orders 2 and 3 BOTH on Jan 2, order 4 on Jan 3) processed two different ways. Left panel (blue background, label 'ROWS — physical row count'): shows 4 sequential boxes with a running total accumulating one row at a time: 100, 300, 550, 850 — each box's value is visibly DIFFERENT from its neighbor even though orders 2 and 3 share the same date, with an annotation: 'Each row counted individually, regardless of tied dates.' Right panel (orange background, label 'RANGE — logical value grouping'): shows the same 4 boxes, but orders 2 and 3 (both Jan 2) are enclosed in a single bracket and BOTH labeled with the identical value 550, with an annotation: 'Rows sharing the SAME order_date are treated as ONE logical unit — both get the same combined total, as if they happened simultaneously.' A center divider reads: 'ROWS and RANGE behave IDENTICALLY when there are no duplicate ORDER BY values. They DIVERGE the moment two rows tie on the ORDER BY column — ROWS still ranks them separately, RANGE merges them into one shared result.' Clean two-panel layout, the Jan 2 rows visually bracketed together in the RANGE panel to emphasize the grouping behavior.

ROWS vs RANGE — Diverging Behavior on Duplicate ORDER BY Values

Student

How do I build something like a 7-day moving average instead of a running total that includes everything from the start?

ChatGPT • AI Assistant

CONCEPT CARD

A **moving window** (also called a sliding window) uses an explicit, **fixed-width** frame instead of the growing **UNBOUNDED PRECEDING** frame. For a 7-day moving average, the frame should include the current row plus the 6 rows immediately before it - 7 rows total:

sql — Code Example

```
SELECT sale_date,
       revenue,
       AVG(revenue) OVER (
         ORDER BY sale_date
         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
       ) AS moving_avg_7day
FROM   daily_sales
ORDER BY sale_date;
```

EXAMPLE CARD

Tracing exactly which rows are included as the frame slides, using a small 10-day sample with a 3-row moving window (`'ROWS BETWEEN 2 PRECEDING AND CURRENT ROW'`) for clarity:

code — Code Example

sale_date	revenue	frame included (3-row window)	
moving_avg_3day			
2024-01-01	100	[Jan 1]	(only 1 row) 100.0
2024-01-02	200	[Jan 1, Jan 2]	(only 2 rows) 150.0
2024-01-03	300	[Jan 1, Jan 2, Jan 3]	(full window) 200.0
2024-01-04	400	[Jan 2, Jan 3, Jan 4]	← Jan 1 DROPPED OUT 300.0
2024-01-05	500	[Jan 3, Jan 4, Jan 5]	← Jan 2 DROPPED OUT 400.0

Notice rows 1 and 2: the frame **cannot** include 2 rows before the current row yet, because those rows do not exist - PostgreSQL and standard SQL simply use however many preceding rows ARE available, rather than erroring or padding with NULLs. This means a 3-row moving average's first two values are technically averages of only 1 and 2 rows respectively, not true 3-row averages - a detail that matters if you are reporting these early values to a user without explanation.

From Jan 3 onward, the window is genuinely always exactly 3 rows wide, but the **specific** rows included **slide forward** by exactly one position each time: Jan 4's frame drops Jan 1 and picks up Jan 4 itself; Jan 5's frame drops Jan 2 and picks up Jan 5. This sliding behavior - fixed width, but the actual set of included rows shifts forward - is the defining characteristic of a moving window, as opposed to a running total's frame, which only ever grows and never drops anything.

Student

You mentioned ROWS and RANGE are different. When does that actually matter?

ChatGPT • AI Assistant

CONCEPT CARD

ROWS counts a fixed number of **physical rows**, regardless of their values. **RANGE** counts based on the **logical value** in the ORDER BY column - and critically, **RANGE treats all rows with the SAME ORDER BY value as belonging to the same point**, including or excluding them together rather than one at a time.

This distinction is invisible and irrelevant when your ORDER BY column has no duplicate values - but becomes important the moment duplicates exist.

EXAMPLE CARD

The exact scenario where ROWS and RANGE diverge - duplicate order_date values:

code – Code Example			
order_id	order_date	total_amount	
1	2024-01-01	100	
2	2024-01-02	200	
3	2024-01-02	250	← SAME date as order 2
4	2024-01-03	300	

sql – Code Example

```
-- Using ROWS: counts physical rows one at a time, ignoring tied dates
SELECT order_id, order_date, total_amount,
       SUM(total_amount) OVER (
         ORDER BY order_date
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_total_rows
FROM   orders_sample;
```

Result with ROWS:

code – Code Example			
order_id	order_date	total_amount	running_total_rows
1	2024-01-01	100	100
2	2024-01-02	200	300
includes only order 1 + order 2			
3	2024-01-02	250	550
includes order 1+2+3			
4	2024-01-03	300	850

sql – Code Example

```
-- Using RANGE: treats BOTH Jan 2 rows as the SAME logical point –
-- both receive the SAME running total, computed as if they happened
together
SELECT order_id, order_date, total_amount,
       SUM(total_amount) OVER (
         ORDER BY order_date
         RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_total_range
```

```
    ) AS running_total_range
FROM    orders_sample;
```

Result with RANGE:

code – Code Example

order_id	order_date	total_amount	running_total_range	
1	2024-01-01	100	100	
2	2024-01-02	200	550	← BOTH
Jan 2 rows get the SAME value				
3	2024-01-02	250	550	←
because RANGE treats tied dates as one unit				
4	2024-01-03	300	850	

With **RANGE** , order 2 and order 3 - tied on **order_date** - **both** receive the running total **as if all rows sharing that date had already been included**, rather than processing them one row at a time. This means with RANGE, a 'running total' on a date with multiple same-day transactions jumps to include ALL of that day's transactions simultaneously for every row sharing that date, rather than accumulating one transaction at a time within the day.

⚠️ WARNING

Most real-world running-total and moving-average use cases want ROWS, not RANGE, precisely because they want a true row-by-row accumulation regardless of tied values, and because **RANGE** with a numeric/fixed-width frame (like **RANGE BETWEEN 6 PRECEDING AND CURRENT ROW**) has different, more complex semantics (it would mean 'all rows within 6 units of the current row's value', not '6 physical rows back') that are rarely what people intend when they say '7-day moving average.'

Always default to writing `ROWS BETWEEN ...` explicitly unless you have a specific, deliberate reason to use the value-based semantics of RANGE.

📖 INTERVIEW CARD

Frequency: High

Question: What is the difference between ROWS and RANGE in a window frame, and when does this difference actually produce different results?

Expected Answer: ROWS counts a fixed number of physical rows in the frame, regardless of their ORDER BY values. RANGE counts based on the logical value of the ORDER BY column, treating all rows that share the same ORDER BY value as a single

logical point - they are all included or excluded together rather than incrementally one row at a time.

This distinction is invisible when the ORDER BY column has no duplicate values, since each row is its own unique logical point either way. It becomes significant the moment duplicates exist: with RANGE, all rows tied on the ORDER BY value receive the identical aggregate result, computed as if the entire tied group had already been processed together, whereas ROWS processes them one physical row at a time, giving each tied row a different incremental result.

Follow-up: For a typical '7-day moving average of daily revenue' use case, should you use ROWS or RANGE, and why? (Answer: ROWS - because the intent is a fixed count of 7 physical rows (days), and daily_sales typically has one row per unique date with no duplicates, making ROWS the simpler, more predictable, and conventionally correct choice; RANGE's value-based semantics are a different and more complex tool intended for genuinely value-driven windowing scenarios, not standard physical-row moving averages.)

🔍 PRACTICE QUIZ

1. With ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW, do two rows with the identical ORDER BY value receive the same running total or different ones?
2. With RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW, do two rows with the identical ORDER BY value receive the same running total or different ones?
3. For the very first two rows of a 3-row moving average (ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), how many rows are actually included in each frame?
4. Which frame type - ROWS or RANGE - should be the default choice for a standard daily moving average?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. Different ones - ROWS processes physical rows one at a time regardless of value ties, so each row in a tied group accumulates incrementally and receives a distinct running total.

2. The same one - RANGE treats all rows sharing the identical ORDER BY value as one logical unit, so every row in that tied group receives the identical aggregate result.
3. Row 1: only 1 row (itself - there are no preceding rows yet). Row 2: only 2 rows (itself and the 1 row before it) - neither row yet has the full 3-row window available.
4. ROWS - it provides the simple, predictable, fixed-physical-row-count behavior that 'moving average' conventionally means, and avoids RANGE's more complex value-based semantics that are rarely the intended behavior for this use case.

🔗 CHECKPOINT QUESTION

You have a sales table with multiple transactions on some days. A colleague writes a 'running total' using RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW and is confused why two transactions on the same day show the identical running total instead of one being slightly higher than the other.

Explain what is happening and rewrite their query using ROWS to fix it.

PRACTICE PROBLEMS — 30-DAY MOVING AVERAGE AND LIFETIME-SPEND THRESHOLD FLAGGING

ChatGPT • AI Assistant

A horizontal sequence of 4 order boxes for one customer, ordered by date, each showing two stacked values: a top bar labeled 'running_lifetime_spend (includes current row)' and a bottom bar labeled 'spend_before_this_order (excludes current row)'. Order 1 (Jan 5): top bar = 3500, bottom bar = 0 (with a small COALESCE icon noting 'NULL converted to 0'). Order 2 (Feb 12): top bar = 7700, bottom bar = 3500. Order 3 (Apr 30, highlighted gold with a star icon): top bar = 10800 (crosses above a horizontal red dashed THRESHOLD LINE at 10000), bottom bar = 7700 (still below the threshold line). An annotation box pointing to Order 3 reads: 'THIS is the crossing order: spend_before (7700) < 10000 AND running_total (10800) >= 10000 — both conditions true ONLY here.' Order 4 (Sep 15): top bar = 12300, bottom bar = 10800 — both bars now above the threshold line, with annotation: 'Already past threshold BEFORE this order even happened — correctly NOT flagged again.' A horizontal red dashed line spans the full width of the diagram at the 10000 mark labeled 'LIFETIME SPEND THRESHOLD.' Clean four-box horizontal layout, dual-bar comparison per box, threshold line clearly crossing only at Order 3's bars.

Isolating the Exact Threshold-Crossing Order Using Two Frames

Student

Let's work through the two practice problems.

ChatGPT • AI Assistant

CONCEPT CARD

Practice Problem 1 - Compute a 30-day moving average of revenue alongside the actual daily value.

sql – Code Example

```
SELECT sale_date,
       revenue,
       ROUND(
         AVG(revenue) OVER (
           ORDER BY sale_date
           ROWS BETWEEN 29 PRECEDING AND CURRENT ROW
         ), 2
       ) AS moving_avg_30day
FROM   daily_sales
ORDER BY sale_date;
```

Why 29 PRECEDING, not 30 PRECEDING: the frame is inclusive of the current row, so to get a total window width of 30 rows, you need 29 rows BEFORE the current row, plus the current row itself, equals 30 rows total. This off-by-one detail is one of the most common mistakes when writing moving-window frames - always double-check: **(N-1) PRECEDING AND CURRENT ROW** gives a window of exactly N rows.

Sample output:

code – Code Example

sale_date	revenue	moving_avg_30day	
2024-01-01	12000	12000.00	← only 1 day of data so far, avg = itself
2024-01-02	12500	12250.00	← avg of 2 days
...			
2024-01-30	13100	12480.33	← first TRUE full 30-day window
2024-01-31	12900	12502.67	← window slides: Jan 1 drops out, Jan 31 joins

ROWS is the correct choice here, not **RANGE**, assuming **daily_sales** has exactly one row per date with no duplicates - the two would behave identically in that case, but **ROWS** is still the conventionally correct and clearer choice to express 'a window of 30 physical day-rows.'

Practice Problem 2 - For each customer, compute running cumulative spend and flag the order on which they crossed a 10,000 lifetime-spend threshold.

sql – Code Example

```

WITH running_spend AS (
    SELECT order_id,
           customer_id,
           order_date,
           total_amount,
           SUM(total_amount) OVER (
               PARTITION BY customer_id
               ORDER BY order_date
               ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
           ) AS running_lifetime_spend,
           -- ALSO compute the spend BEFORE this order, using LAG-
           style logic
           -- via a one-row-shorter frame (excludes CURRENT ROW)
           SUM(total_amount) OVER (
               PARTITION BY customer_id
               ORDER BY order_date
               ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING
           ) AS spend_before_this_order
    FROM   orders
    WHERE  status ≠ 'cancelled'
)
SELECT order_id,
       customer_id,
       order_date,
       total_amount,
       running_lifetime_spend,
       COALESCE(spend_before_this_order, 0) AS
       spend_before_this_order,
       CASE
           WHEN COALESCE(spend_before_this_order, 0) < 10000
                AND running_lifetime_spend ≥ 10000
           THEN TRUE
           ELSE FALSE
       END AS crossed_10k_threshold_here
FROM   running_spend
ORDER BY customer_id, order_date;

```

Sample output for one customer:

code – Code Example

```

order_id | order_date | total_amount | running_lifetime_spend |
spend_before_this_order | crossed_10k_threshold_here

```

5001	2024-01-05	3500.00	3500.00	0.00
FALSE				
5034	2024-02-12	4200.00	7700.00	
3500.00		FALSE		
5210	2024-04-30	3100.00	10800.00	
7700.00		TRUE	← crossed here!	
5402	2024-09-15	1500.00	12300.00	
10800.00		FALSE	← already past 10k	

The key mechanism: `spend_before_this_order` uses `ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING` - a frame that **excludes the current row entirely**, ending one row earlier than the running total's frame. Comparing this 'spend before' value (still under 10,000) against the regular running total including the current order (now at or over 10,000) precisely identifies the **exact transaction** where the customer's cumulative lifetime spend crossed the threshold - not just every row after they happened to already be above it.

⚠ WARNING

`COALESCE(spend_before_this_order, 0)` is necessary because the very first order for any customer has **no rows before it at all** - the frame `UNBOUNDED PRECEDING AND 1 PRECEDING` produces NULL for that first row (an empty frame), and NULL compared against any number using `<` would itself evaluate to NULL, silently making the entire CASE expression's first condition fail to register as TRUE even on a customer whose very first order is itself over 10,000. Wrapping it in `COALESCE` converts that empty-frame NULL into a clean 0, allowing the threshold comparison to work correctly even on a customer's first-ever order.

📌 INTERVIEW CARD

Frequency: High

Question: How would you identify the exact transaction where a customer's cumulative spend first crosses a specific threshold?

Expected Answer: Compute two window aggregates: the running total **INCLUDING** the current row (standard `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`), and the running total **EXCLUDING** the current row (`ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING`, which only sums rows strictly before the current one). The threshold-crossing row is precisely the one where the 'excluding current row' total is still below the threshold but the 'including current row' total is at or above it - this isolates the single transaction responsible for

tipping the cumulative total over the line, rather than every row that happens to already be above the threshold.

The first row of each partition needs a COALESCE to handle the empty frame, which otherwise produces NULL for the 'before this order' value.

Follow-up: Why is COALESCE specifically needed for the FIRST row of each customer's partition in this calculation? (Answer: The frame ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING for the very first row of a partition refers to rows that do not exist - there is nothing before the first row - producing an empty frame, which SUM evaluates as NULL rather than 0.

Without COALESCE, comparing `NULL < 10000` evaluates to NULL, not TRUE, which would incorrectly prevent the threshold-crossing flag from ever firing on a customer whose very first order alone exceeds the threshold.)

🔗 PRACTICE QUIZ

1. In the 30-day moving average, why is the frame written as 29 PRECEDING rather than 30 PRECEDING?
2. What frame definition computes 'the running total NOT including the current row'?
3. Why does the threshold-crossing CASE expression compare `spendbeforethisorder < 10000 AND runninglifetimespend >= 10000`, rather than just checking `runninglifetime_spend >= 10000` alone?
4. What problem does COALESCE solve in the threshold-crossing query?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. Because the frame is inclusive of the current row - 29 rows before plus the current row itself equals a total window width of exactly 30 rows. Writing 30 PRECEDING would produce a 31-row window.
2. `ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING` - this frame ends one row before the current row, explicitly excluding it.
3. Because `running_lifetime_spend ≥ 10000` alone is TRUE for every row from the crossing point onward, not just the one specific row where the crossing actually happened - adding the condition that the PRIOR total was still below 10000 isolates the exact single transaction responsible for the crossing.

4. It converts the NULL produced by an empty frame (on each customer's very first order, where no 'preceding rows' exist) into 0, so the threshold comparison works correctly even for a customer whose first order alone exceeds the threshold.

🔍 CHECKPOINT QUESTION

Before concluding this topic: rewrite the threshold-crossing query to instead flag the order where a customer's running ORDER COUNT (not spend) first reaches exactly 5 orders - i.e., flag their 5th order specifically. Explain whether you still need both an 'including current row' and 'excluding current row' window for this simpler count-based version, or whether a single window function would suffice.

SUMMARY CARD

SQL 1.14 Key Takeaways:

- **LAG(column, offset, default)** reaches backward N rows (default offset=1) within a partition; returns the specified default (or NULL) at partition boundaries where no such row exists.
- **LEAD(column, offset, default)** is the mirror image, reaching forward - a NULL from LEAD often correctly signals 'this is the last row in this partition' (e.g., a customer's most recent order).
- **The window frame clause** activates the moment ORDER BY is added inside OVER() - the default frame **RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW** is exactly what produces a running total.
- **Without ORDER BY**, an aggregate window function defaults to the WHOLE partition for every row - the same grand total repeated everywhere, not a running total.
- **Moving windows** use a fixed-width explicit frame like **ROWS BETWEEN 6 PRECEDING AND CURRENT ROW** for a 7-row window; remember the off-by-one rule: **(N-1) PRECEDING AND CURRENT ROW** gives exactly N rows.
- **ROWS** counts physical rows one at a time. **RANGE** treats rows tied on the ORDER BY value as one logical unit, giving them identical results. They diverge only when duplicate ORDER BY values exist - default to ROWS unless you specifically need value-based grouping.
- **PARTITION BY** resets both running totals and moving windows per group, exactly as it resets ranking functions.
- **Excluding the current row from a frame** (**... AND 1 PRECEDING**) is the technique for comparing 'before this row' against 'including this row' - essential for precisely isolating threshold-crossing events.

🌟 AI DEEP DIVE PROMPT

Copy this prompt into Claude or ChatGPT for deeper practice:

'Act as a senior SQL mentor. I just learned LAG, LEAD, running totals, moving averages, and the ROWS vs RANGE frame distinction.

Give me 8 progressively harder exercises using realistic business scenarios - period-over-period growth, churn gap detection, cumulative revenue charts, moving averages with duplicate dates, and threshold-crossing detection. For each exercise: give me the table schema and a small sample dataset, ask me to trace through the expected output by hand BEFORE writing SQL, then ask me to write the query, then show the correct solution highlighting any off-by-one frame mistakes or NULL-handling pitfalls.

Do not continue until I submit my answer.'

🕒 CHECKPOINT QUESTION

Final review for this topic: a dashboard needs to show, for every day, the actual daily revenue, the 7-day moving average, AND the day-over-day percentage change versus the SAME day last week (7 days ago) - all three values in one query. Identify which window function or frame technique each of the three columns requires, and write the complete query combining all of them.